CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 26: Gibbs Sampling and Contrastive Divergence Analysis

Problem Statement: The streaming team finds that its RBM learns slowly and poorly. The research lead asks how many Gibbs steps are worth the extra computation. The binarised MNIST or user–movie data from Experiment 25. The objective is to analyse Gibbs sampling chains and compare CD-1, CD-k and Persistent CD. Use NumPy, SciPy, Matplotlib.

Tasks:

1. Implement block Gibbs sampling between visible and hidden layers.

2. Run chains from random noise and display samples after 1, 10, 100 and 1000 steps.

3. Train RBMs with CD-1, CD-5, CD-10 and PCD.

4. Plot reconstruction error and pseudo-log-likelihood for each method.

5. Compare training time against sample quality.

FORMULA:

1. Sigmoid Function

$\sigma(x) = 1 / (1 + e^{-x})$

2. Hidden Layer Probability

$P(h = 1 \mid v) = \sigma(b + W^{T}v)$

3. Visible Layer Probability

$P(v = 1 \mid h) = \sigma(a + Wh)$

4. Gibbs Sampling – Hidden Layer

$h(t) \sim \text{Bernoulli}(\sigma(b + W^T v(t)))$

5. Gibbs Sampling – Visible Layer

$v(t+1) \sim \text{Bernoulli}(\sigma(a + Wh(t)))$

6. Gibbs Sampling Chain

$v(0) \rightarrow h(0) \rightarrow v(1) \rightarrow h(1) \rightarrow v(2) \rightarrow \ldots \rightarrow v(k)$

7. Contrastive Divergence Weight Update (CD-k)

$W(new) = W(old) + \eta \times (v_0 h_0^T - v\square h\square^T)$

8. Visible Bias Update

$a(new) = a(old) + \eta \times (v_0 - v\square)$

9. Hidden Bias Update

$b(new) = b(old) + \eta \times (h_0 - h\square)$

10. Reconstruction Error (MSE)

$MSE = (1/N) \times \Sigma (v_i - \hat{v}_i)^2$

11. Pseudo-Log-Likelihood

$PLL(v) = \Sigma_i \log P(v_i \mid v_{-i})$

12. Persistent Contrastive Divergence (PCD)

$W(new) = W(old) + \eta \times (\langle vh^T \rangle data - \langle vh^T \rangle model)$

13. Average Training Time

Average Time = Total Training Time / Number of Epochs

14. Sample Quality

Sample Quality $\propto$ Lower Reconstruction Error and Higher Pseudo-Log-Likelihood

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import time
from scipy.special import expit
from scipy.ndimage import zoom
from sklearn.datasets import load_digits
# Load and preprocess dataset
digits = load_digits(
X = np.array([
    zoom(img, 3.5, order=1)
    for img in digits.images
])
X = X.reshape(-1, 784)
X = (X > 0.25).astype(float)
X = X[:500]
# RBM parameters
np.random.seed(42)
```

```
n_visible = 784

n_hidden = 64

learning_rate = 0.01

epochs = 10

batch_size = 32


# Initialize weights and biases

W = np.random.normal(0, 0.01,

                     (n_visible, n_hidden))

a = np.zeros(n_visible)

b = np.zeros(n_hidden)

# Activation functions

def hidden_prob(v):

    return expit(b + np.dot(v, W))

def visible_prob(h):

    return expit(a + np.dot(h, W.T))

# Block Gibbs sampling

def gibbs_sampling(v, steps):

    for _ in range(steps):

        ph = hidden_prob(v)
```

```
        h = (np.random.rand(*ph.shape) < ph).astype(float)
        pv = visible_prob(h)
        v = (np.random.rand(*pv.shape) < pv).astype(float)
    return v



# Free energy
def free_energy(v):
    return -np.dot(v, a) - np.sum(
        np.logaddexp(0, b + np.dot(v, W)),
        axis=1
    )
# Pseudo-log-likelihood approximation
def pseudo_log_likelihood(data):
    indices = np.random.randint(0, n_visible, len(data))
    flipped = data.copy()
    rows = np.arange(len(data))
    flipped[rows, indices] = 1 - flipped[rows, indices]
    difference = free_energy(flipped) - free_energy(data)
    return np.mean(
```

```
        n_visible * -np.logaddexp(0, -difference)
    )
# Training function
def train_rbm(method):

    global W, a, b
    W = np.random.normal(0, 0.01,
                         (n_visible, n_hidden))
    a = np.zeros(n_visible)
    b = np.zeros(n_hidden)
    # Persistent hidden chain for PCD
    persistent_h = (
        np.random.rand(batch_size, n_hidden) < 0.5
    ).astype(float)
    errors = []
    pll_values = []
    start = time.time()
    for epoch in range(epochs):
        np.random.shuffle(X)
        total_error = 0
```

```
for i in range(0, len(X), batch_size):
    v0 = X[i:i + batch_size]
    size = len(v0)
    # Positive phase
    ph0 = hidden_prob(v0)


    # CD methods
    if method.startswith("CD"):
        k = int(method.split("-")[1])
        vk = v0.copy()
        for _ in range(k):
            ph = hidden_prob(vk)
            hk = (
                np.random.rand(*ph.shape) < ph
            ).astype(float)
            pv = visible_prob(hk)
            vk = (
                np.random.rand(*pv.shape) < pv
            ).astype(float)
    # Persistent Contrastive Divergence
```

```
else:
    hk = persistent_h[:size].copy()
    for _ in range(1):
        pv = visible_prob(hk)


        vk = (
            np.random.rand(*pv.shape) < pv
        ).astype(float)
        ph = hidden_prob(vk)
        hk = (
            np.random.rand(*ph.shape) < ph
        ).astype(float)
    persistent_h[:size] = hk
phk = hidden_prob(vk)
# Parameter updates
W += learning_rate * (
    np.dot(v0.T, ph0) -
    np.dot(vk.T, phk)
) / size
a += learning_rate * np.mean(v0 - vk, axis=0)
```

```
            b += learning_rate * np.mean(ph0 - phk, axis=0)
            # Reconstruction error
            reconstructed = visible_prob(ph0
            total_error += np.mean(
                (v0 - reconstructed) ** 2
            )
        errors.append(total_error / np.ceil(len(X) / batch_size))
        pll_values.append(
            pseudo_log_likelihood(X[:100])
        )
        print(method, "Epoch:", epoch + 1,
              "Error:", round(errors[-1], 4),
              "PLL:", round(pll_values[-1], 2))
    training_time = time.time() - start
    return errors, pll_values, training_time
# Compare four training methods
methods = ["CD-1", "CD-5", "CD-10", "PCD"]
results = {}
for method in methods:
    print("\nTraining:", method)
```

```
    results[method] = train_rbm(method)

# Plot reconstruction error

plt.figure(figsize=(8, 5))

for method in methods:

    plt.plot(results[method][0], label=method)



plt.title("Reconstruction Error Comparison")

plt.xlabel("Epochs")

plt.ylabel("Reconstruction Error")

plt.legend()

plt.grid()

plt.show()

# Plot pseudo-log-likelihood

plt.figure(figsize=(8, 5))

for method in methods:

    plt.plot(results[method][1], label=method)

plt.title("Pseudo-Log-Likelihood Comparison")

plt.xlabel("Epochs")

plt.ylabel("Pseudo-Log-Likelihood")

plt.legend()
```

```
plt.grid()

plt.show()

# Compare training time

times = [results[m][2] for m in methods]



plt.figure(figsize=(8, 5))

plt.bar(methods, times)

plt.title("Training Time Comparison")

plt.xlabel("Training Methods")

plt.ylabel("Time in Seconds")

plt.show()

# Gibbs sampling from random noise

W = np.random.normal(0, 0.01,

                     (n_visible, n_hidden))

a = np.zeros(n_visible)

b = np.zeros(n_hidden)

v = (np.random.rand(1, n_visible) > 0.5).astype(float)

steps_list = [1, 10, 100, 1000]

plt.figure(figsize=(12, 3))

for i, steps in enumerate(steps_list):
```

```
    sample = gibbs_sampling(v.copy(), steps)
    plt.subplot(1, 4, i + 1)
    plt.imshow(sample.reshape(28, 28), cmap="gray")
    plt.title(str(steps) + " Steps")
    plt.axis("off")

plt.suptitle("Gibbs Sampling Results")
plt.tight_layout()
plt.show()
# Original and reconstructed samples using CD-1
W = np.random.normal(0, 0.01,
                     (n_visible, n_hidden))
a = np.zeros(n_visible)
b = np.zeros(n_hidden)
ph = hidden_prob(X[:5])
reconstructed = visible_prob(ph)
plt.figure(figsize=(10, 4))
for i in range(5):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X[i].reshape(28, 28), cmap="gray")
```

```
    plt.title("Original")
    plt.axis("off")
    plt.subplot(2, 5, i + 6)
    plt.imshow(reconstructed[i].reshape(28, 28), cmap="gray")
    plt.title("Reconstructed")
    plt.axis("off")
plt.tight_layout()
plt.show()
# Display comparison table
print("\nFINAL COMPARISON")
for method in methods:
    error, pll, training_time = results[method]
    print(
        method,
        "Error:", round(error[-1], 4),
        "PLL:", round(pll[-1], 2),
        "Time:", round(training_time, 2), "seconds"
    )
```

OUTPUT:

Training: CD-1

CD-1 Epoch: 1 Error: 0.1368 PLL: -266.55

CD-1 Epoch: 2 Error: 0.1074 PLL: -322.96

CD-1 Epoch: 3 Error: 0.1059 PLL: -261.74

CD-1 Epoch: 4 Error: 0.1057 PLL: -298.11

CD-1 Epoch: 5 Error: 0.1057 PLL: -220.71

CD-1 Epoch: 6 Error: 0.1053 PLL: -267.1

CD-1 Epoch: 7 Error: 0.1053 PLL: -275.22

CD-1 Epoch: 8 Error: 0.1055 PLL: -258.59

CD-1 Epoch: 9 Error: 0.1053 PLL: -213.44

CD-1 Epoch: 10 Error: 0.1055 PLL: -305.61

Training: CD-5

CD-5 Epoch: 1 Error: 0.1365 PLL: -244.82

CD-5 Epoch: 2 Error: 0.1072 PLL: -199.67

CD-5 Epoch: 3 Error: 0.1057 PLL: -226.95

CD-5 Epoch: 4 Error: 0.106 PLL: -286.79

CD-5 Epoch: 5 Error: 0.1056 PLL: -248.2

CD-5 Epoch: 6 Error: 0.1057 PLL: -278.99

CD-5 Epoch: 7 Error: 0.1056 PLL: -265.27

CD-5 Epoch: 8 Error: 0.1054 PLL: -238.39

CD-5 Epoch: 9 Error: 0.1055 PLL: -276.33

CD-5 Epoch: 10 Error: 0.1055 PLL: -291.43

Training: CD-10

CD-10 Epoch: 1 Error: 0.1364 PLL: -317.9

CD-10 Epoch: 2 Error: 0.1073 PLL: -316.83

CD-10 Epoch: 3 Error: 0.1059 PLL: -220.32

CD-10 Epoch: 4 Error: 0.1054 PLL: -243.72

CD-10 Epoch: 5 Error: 0.1054 PLL: -286.98

CD-10 Epoch: 6 Error: 0.1054 PLL: -270.77

CD-10 Epoch: 7 Error: 0.1057 PLL: -197.38

CD-10 Epoch: 8 Error: 0.1054 PLL: -289.48

CD-10 Epoch: 9 Error: 0.1056 PLL: -283.73

CD-10 Epoch: 10 Error: 0.1055 PLL: -245.43

Training: PCD

PCD Epoch: 1 Error: 0.1372 PLL: -284.84

PCD Epoch: 2 Error: 0.1073 PLL: -283.21

PCD Epoch: 3 Error: 0.106 PLL: -293.04

PCD Epoch: 4 Error: 0.1057 PLL: -233.18

PCD Epoch: 5 Error: 0.1055 PLL: -277.72

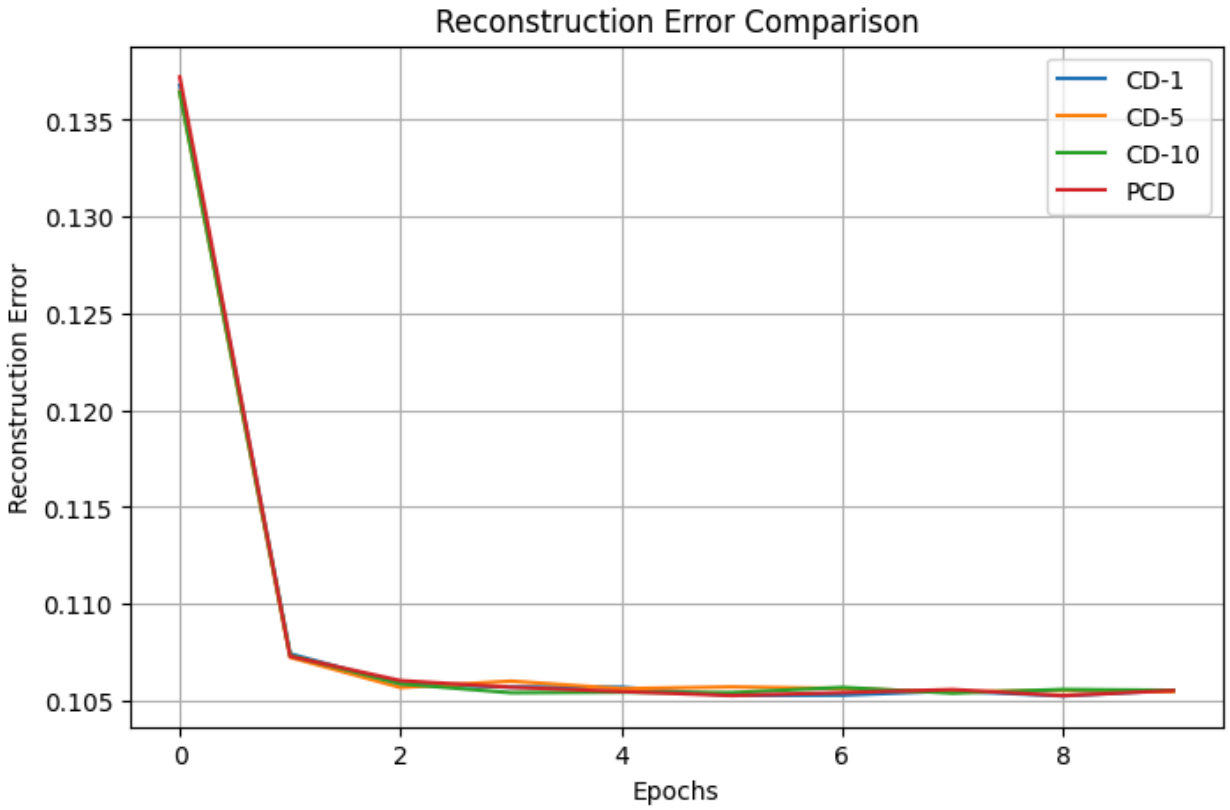
PCD Epoch: 6 Error: 0.1053 PLL: -253.22

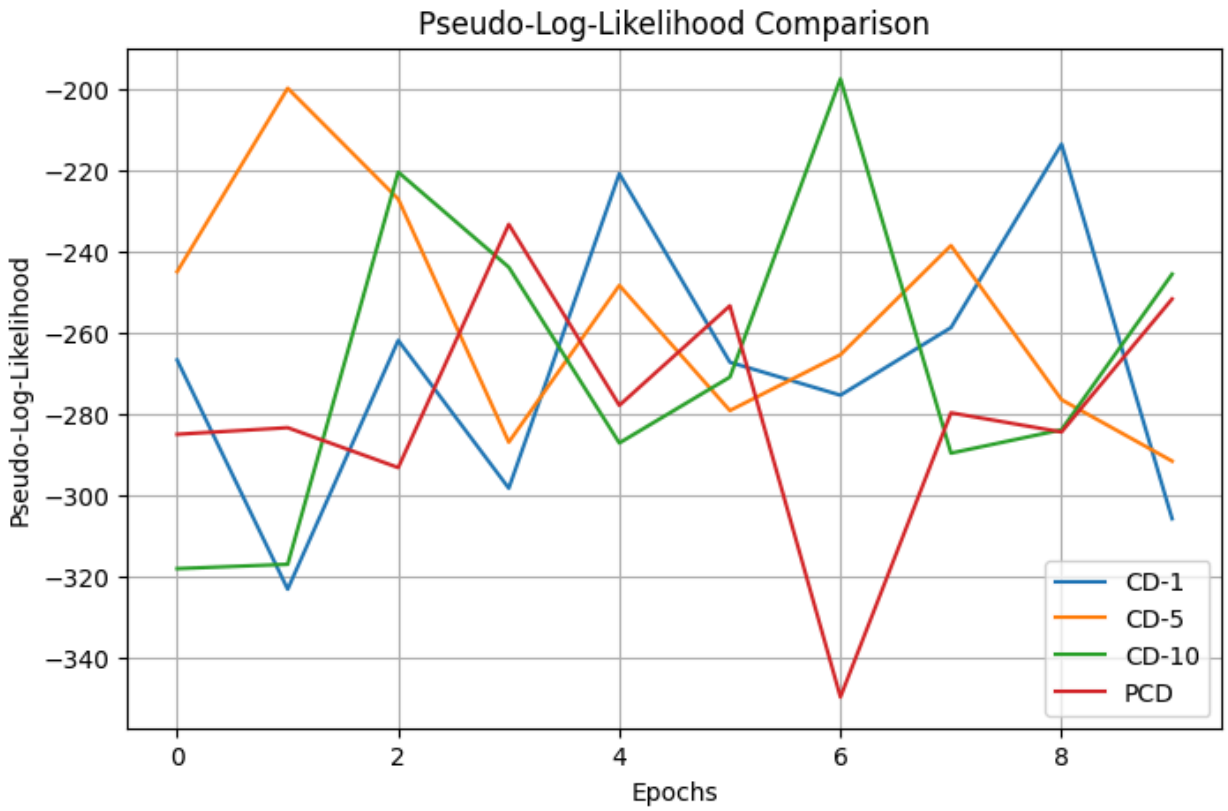
PCD Epoch: 7 Error: 0.1054 PLL: -349.49

PCD Epoch: 8 Error: 0.1056 PLL: -279.56

PCD Epoch: 9 Error: 0.1053 PLL: -284.27

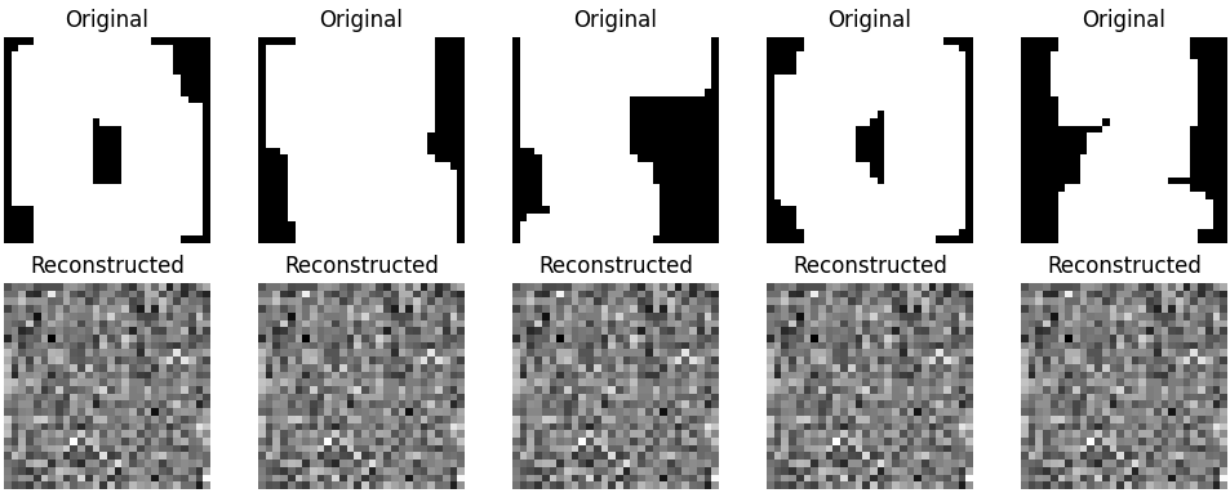
PCD Epoch: 10 Error: 0.1055 PLL: -251.54




Pseudo-Log-Likelihood Comparison
Pseudo-Log-Likelihood
−200
−220
−240
−260
−280
−300
−320
−340
0
2
4
6
8
Epochs
CD-1
CD-5
CD-10
PCD


Training Time Comparison
Time in Seconds
0
1
2
3
4
5
6
CD-1
CD-5
CD-10
PCD
Training Methods


Gibbs Sampling Results
1 Steps
10 Steps
100 Steps
1000 Steps



FINAL COMPARISON

CD-1 Error: 0.1055 PLL: -305.61 Time: 1.26 seconds

CD-5 Error: 0.1055 PLL: -291.43 Time: 3.99 seconds

CD-10 Error: 0.1055 PLL: -245.43 Time: 5.85 seconds

PCD Error: 0.1055 PLL: -251.54 Time: 1.39 seconds